# Practice Interview

## Objective

**The partner assignment aims to provide participants with the opportunity to practice coding in an interview context. You will analyze your partner's Assignment 1. Moreover, code reviews are common practice in a software development team. This assignment should give you a taste of the code review process.**

## Group Size

Each group should have 2 people. You will be assigned a partner

## Part 1:

You and your partner must share each other's Assignment 1 submission.

## Part 2:

Create a Jupyter Notebook, create 6 of the following headings, and complete the following for your partner's assignment 1:

- Paraphrase the problem in your own words.

In [ ]:
```
# Your answer here
# Given a tree, with the input in the form of level order traversal method.
# Return all combinations of root to leaf paths.
# The output can be in any order.
```

- Create 1 new example that demonstrates you understand the problem. Trace/walkthrough 1 example that your partner made and explain it.

In [ ]:
```
# Your answer here
# Manual example:
# Input: [3, 4, 2, 5, 6, 2, 3, 3, 1]
# Output: [[3, 4, 5, 3], [3, 4, 5, 1], [3, 4, 6], [3, 2, 2], [3, 2, 3]]

#Example1 - From Parisa's assignment
root = TreeNode(5)
root.left = TreeNode(3)
root.right = TreeNode(8)
root.left.left = TreeNode(1)
root.left.right = TreeNode(4)

print(bt_path(root))

# In this approach, each value of the tree is defined through the specific node they they belong to, if using the sam

# The output was achieved by running the input through the program, and printing the result, which is: [[5, 3, 1], [5
```

```
[[5, 3, 1], [5, 3, 4], [5, 8]]
```

- Copy the solution your partner wrote.

In [1]:
```
# Your answer here

# Definition for a binary tree node.
class TreeNode(object):
     def __init__(self, val = 0, left = None, right = None):
         self.val = val
         self.left = left
         self.right = right
def bt_path(root: TreeNode) -> list[list[int]]:
  def dfs(node, path, paths):
        if not node:
            return
        path.append(node.val)

        if not node.left and not node.right:
            paths.append(list(path))
        else:
            dfs(node.left, path, paths)
```

```
            dfs(node.right, path, paths)

        path.pop()

    paths = []
    dfs(root, [], paths)
    return paths
```

- Explain why their solution works in your own words.

In [ ]:
```
# Your answer here
# The solution works through recursion.
# The function is given a TreeNode as input, and the dfs function would recursively
# access each element of the node going deeper and deeper with each pass,
# the dfs function also takes in two additional parameters, path and paths.
# path is initialized with an empty list, and paths is just defined as a variable.
# As the program goes down through each path, the value of the node is stored in
# path, and once it has gone through the last element, path is appended to paths.
# After going through all possible paths, the function would then return paths
# which contain the final output.
```

- Explain the problem's time and space complexity in your own words.

In [ ]:
```
# Your answer here
# Time complexity:
# Worst case: O(n log n)
# The worst case would be by having a balanced binary tree as the input,
# where each node would have a value, each pass, halving the number of elements left.
# Best case: O(n)
# The best case is where we have a skewed binary tree, where in only one side is
# filled, this would be essentially traversing through n elements O(n)
#
# Space complexity:
# Best case: O(n)
# In a skewed tree, there would be O(n) space complexity, to traverse through one path
# Worst case: O(n log n)
# In a balanced tree:
# There would be O(n/2) paths, which could be simplified to O(n)
# Plus the space of the call stack which would be the height O(h) where h is log n
```

- Critique your partner's solution, including explanation, and if there is anything that should be adjusted.

In [ ]:
```
# Your answer here
# The solution works well, given the recursive nature of the Tree data structure,
# having a recursive function as a solution works well.
# The solution also possibly achieved the best theoretical time complexity.
```

# Part 3:

Please write a 200 word reflection documenting your process from assignment 1, and your presentation and review experience with your partner at the bottom of the Jupyter Notebook under a new heading "Reflection." Again, export this Notebook as pdf.

## Reflection

In [ ]:
```
# Your answer here
```

For this assignment, I followed a structured approach to understand, analyze, and evaluate the problem assigned to my classmate.

I began by thoroughly reading the problem statement to ensure I fully understood its requirements, objectives, and potential challenges. This initial step was essential for establishing context and preparing for deeper analysis. Following this, I conducted independent research on the problem, exploring solutions and techniques as though I were solving it myself. This included reviewing theoretical approaches, potential algorithms, and best-case complexities for the problem. This step allowed me to build a solid foundation of knowledge, which would help in analyzing my classmate's work.

Once I felt confident in my understanding of the problem, I moved on to review my classmate's solution. I carefully read through their implementation, examining their logic, code structure, and the efficiency of their approach. I paid close attention to their handling of edge cases and analyzed the strengths and areas where improvements might be suggested.

Finally, I compared their solution against the theoretical best-case scenarios I had researched, focusing on time and space complexity. This step helped me evaluate how optimal their solution was and identify areas for refinement. This reflective process enhanced both my problem-solving and critical analysis skills.

## Evaluation Criteria

We are looking for the similar points as Assignment 1

- Problem is accurately stated
- New example is correct and easily understandable
- Correctness, time, and space complexity of the coding solution
- Clarity in explaining why the solution works, its time and space complexity
- Quality of critique of your partner's assignment, if necessary

## Submission Information

🚨 **Please review our Assignment Submission Guide** 🚨 for detailed instructions on how to format, branch, and submit your work. Following these guidelines is crucial for your submissions to be evaluated correctly.

## Submission Parameters:

- Submission Due Date: `HH:MM AM/PM - DD/MM/YYYY`
- The branch name for your repo should be: `assignment-2`
- What to submit for this assignment:
  - This Jupyter Notebook (assignment_2.ipynb) should be populated and should be the only change in your pull request.
- What the pull request link should look like for this assignment: `https://github.com/<your_github_username>/algorithms_and_data_structures/pull/<pr_id>`
  - Open a private window in your browser. Copy and paste the link to your pull request into the address bar. Make sure you can see your pull request properly. This helps the technical facilitator and learning support staff review your submission easily.

Checklist:

- [ ] Created a branch with the correct naming convention.
- [ ] Ensured that the repository is public.
- [ ] Reviewed the PR description guidelines and adhered to them.
- [ ] Verify that the link is accessible in a private browser window.

If you encounter any difficulties or have questions, please don't hesitate to reach out to our team via our Slack at `#cohort-3-help`. Our Technical Facilitators and Learning Support staff are here to help you navigate any challenges.